Document number	P3156
Date	2024-02-15
Audience	LEWG, SG9 (Ranges)

Reply-to	Hewill Kang <hewillk@gmail.com>

`empty_checkable_range`

Abstract

This paper proposes a new concept `empty_checkable_range` that is orthogonal to `sized_range`, to ensure that a range can be checked for emptyness in constant time. In addition, *input* range adaptors have also been enhanced to have an `empty()` member (if possible).

Revision history

R0

Initial revision.

Discussion

What's the issue?

Currently, except for `ref_view` and `owning_view`, which explicitly provide the constrained `empty()` member, other range adaptors do not provide `empty()`. This may be because the original design expected these range adaptors to get free `empty()` from the `view_interface`.

However, `view_interface::empty()` requires derived classes to satisfy `forward_range` or `sized_range`, which ultimately results in non-sized input ranges with an valid `empty()` member losing the functionality to query empty when applied to these adaptors. A real-life example would be:

```
std::istringstream ints("1 2 3 4 5");
auto s = std::ranges::subrange(std::istream_iterator<int>(ints),
                               std::istream_iterator<int>());
std::println("{}", s.empty());
auto r = s | std::views::transform([](int i) { return i * i; })
std::println("{}", r.empty()); // failed
```

`views::transform` does not change the number of elements of the underlying range, so there is no reason not to preserve the original range's ability to check for emptiness. The author believes that for most other adaptors that support input ranges, such as `views::as_rvalue` or `views::enumerate`, etc., this can be done by checking if `ranges::empty(base_)` is valid.

Another point worth noting is that the current standard does not specify the meaning of `ranges::empty` nor its time complexity in the same way that `sized_range` does for `ranges::size`. This makes although `ranges::empty(r)` appears to be valid, the value returned is not required to be the number of elements in range. For example `r.empty()` might be similar to `vector::clear()` for the purpose of clearing the elements with some custom range type.

This implies that it is necessary to introduce the concept corresponding to `ranges::empty` to specify its semantics when applied to ranges.

The `empty_checkable_range` concept

Currently, the `empty()` of `ref_view` and `owning_view` have the following similar definitions:

```
constexpr bool empty() requires requires { ranges::empty(r_); }
{ return ranges::empty(r_); }
constexpr bool empty() const requires requires { ranges::empty(r_); }
{ return ranges::empty(r_); }
```

In other words, we only require that `ranges::empty` can act on the member, but we do not explain the meaning of its return value, nor do we even require its time complexity. Even `const R` may not be a `range`.

In order to clearly specify semantic requirements of `ranges::empty` in the ranges world, this paper introduces a new concept named `empty_checkable_range` similar to `sized_range`, which basically has the following definition:

```
template<class T>
concept empty_checkable_range = range<T> && requires(T& t) { ranges::empty(t); };
```

and accompanied by time complexity requirements. With the help of the new concept, we can respell these empty members as:

```
constexpr bool empty() requires empty_checkable_range<R>
{ return ranges::empty(r_); }
constexpr bool empty() const requires empty_checkable_range<const R>
{ return ranges::empty(r_); }
```

If the user does not want to check the emptiness through the `empty()` member, they can bypass the call by specializing `disable_empty_checkable_range` and check in other meaningful ways, just like `disable_sized_range` does.

Replace existing constraints with `empty_checkable_range`

The new `empty_checkable_range` now can be used to replace members that currently only constrain `ranges::empty` to well-formed to improve semantics, such as `view_interface::operator bool()` as well as `ref_view::empty()` and `owning_view::empty()`.

Provides the constrained `empty()` for the non-forward range adaptors

Since `view_interface::empty()` can always be effectively synthesized by `ranges::begin(`*`derived`*`()) == ranges::end(`*`derived`*`())` when the derived class satisfies `forward_range`, this paper only provides `empty()` for *input* range adaptors.

It should be noted that there are two places where `ranges::empty` is applied to the forward range to check whether it is empty, namely `split_view::`*`find-next`*`()` and `cartesian_product_view::end()`. Although `ranges::empty` is always a valid expression, the semantics of `r.empty()` are not required in those cases, the paper prefers to use the formula of `ranges::begin(r) == ranges::end(r)` to perform the check.

Adaptors that do not change the number of elements

For adaptors that do not change the number of elements, the `empty()` member can be provided when the underlying range models `empty_checkable_range`, by simply returning `ranges::empty(`*`base_`*`)`.

This category includes `as_rvalue_view`, `transform_view`, `common_view`, `as_const_view`, `elements_view` and `enumerate_view`.

Adatoprs that will change the number of elements

There are four input adaptors that change the number of elements and be capable of providing `empty()` worth discussing, namely `take_view`, `lazy_split_view`, `chunk_view` and `stride_view`.

1. The formula for `empty()` member of `take_view` is relatively trivial: as long as the underlying range is empty, it is always empty; otherwise it is empty when `n` is `0`.
2. `lazy_split_view` is only when base and pattern range are empty after LWG 4017 (if accepted).
3. Since the latter two have *Preconditions* of `n > 0`, they will only be empty when the underlying range is empty.

Adatoprs that containing multiple underlying ranges

For input adaptors that contain multiple ranges such as `zip_view` and `cartesian_product_view`, the adaptors will be empty when one of the underlying ranges is empty, so its `empty()` can simply return a logical *OR* of `ranges::empty` applies to all underlying ranges. By analogy, the formula for C++26 `concat_view::empty()` can be spelled as logical *ALL* for all the `ranges::empty` values.

It should be noted that although `cartesian_product_view` except the first range, the rest ranges must model `forward_range`, this means that they can all be used to check whether there are no elements by comparing the return values of `begin()` and `end()`. However, this is no more efficient than calling its `empty()` member directly, so the author prefers to further constrain these forward ranges to `empty_checkable_range` in favor of checking through the `ranges::empty`.

Do we need to relax the constraints of `view_interface::empty()`?

Since the paper provides all input range adaptors with an `empty()` member that constrains the underlying range to `empty_checkable_range`, this makes `view_interface::empty()` constraining the derived class to `sized_range` no longer seems necessary. This is because `ranges::empty` checks `ranges::size(r) == 0` when `r.empty()` is not a valid expression, so this is already compatible with all input-sized range cases.

However, the author believes that checking whether the range is empty through `ranges::size` is still more efficient than constructing iterator-sentinel pair through `ranges::begin` and `ranges::end`. For this reason, the author does not intend to touch the existing signature of `view_interface::empty()`.

Proposed change

This wording is relative to N4971.

1. Modify 26.2 [ranges.syn], Header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]

namespace std::ranges {
  […]
  // [range.sized], sized ranges
  template<class>
    constexpr bool disable_sized_range = false;                                 // freestanding

  template<class T>
    concept sized_range = see below;                                            // freestanding


  // [range.empty.checkable], empty checkable ranges
  template<class>
    constexpr bool disable_empty_checkable_range = false;                       // freestanding

  template<class T>
    concept empty_checkable_range = see below;                                  // freestanding
  […]
}
```

2. Modify 26.3.12 [range.prim.empty] as indicated:

-1- The name `ranges::empty` denotes a customization point object ([customization.point.object]).

-2- Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:

(2.1) — If `T` is an array of unknown bound ([dcl.array]), `ranges::empty(E)` is ill-formed.

(2.2) — Otherwise, if `disable_empty_checkable_range<remove_cv_t<T>>` ([range.empty.checkable]) is `false` and `bool(t.empty())` is a valid expression, `ranges::empty(E)` is expression-equivalent to `bool(t.empty())`.

[…]

3. Add **Empty checkable ranges** [range.empty.checkable] after 26.4.3 [range.sized] as indicated:

-1- The `empty_checkable_range` concept refines `range` with the requirement that `ranges::empty` can be used to check that there are no elements in the range in amortized constant time.

```
template<class T>
  concept empty_checkable_range =
    range<T> && requires(T& t) { ranges::empty(t); };
```

-2- Given an lvalue `t` of type `remove_reference_t<T>`, `T` models `empty_checkable_range` only if

(2.1) — `ranges::empty(t)` is amortized $\mathcal{O}(1)$, does not modify `t`, and is equal to `ranges::distance(ranges::begin(t), ranges::end(t)) == 0`, and

(2.2) — if `iterator_t<T>` models `forward_iterator`, `ranges::empty(t)` is well-defined regardless of the evaluation of `ranges::begin(t)`.

[*Note 1*: `ranges::empty(t)` is otherwise not required to be well-defined after evaluating `ranges::begin(t)`. For example, it is possible for `ranges::empty(t)` to be well-defined for an `empty_checkable_range` whose iterator type does not model `forward_iterator` only if evaluated before the first call to `ranges::begin(t)`. — *end note*]

4. Modify 26.5.3.1 [view.interface.general], class template `view_interface` synopsis, as indicated:

```
namespace std::ranges {
  template<D>
    requires is_class_v<D> && same_as<D, remove_cv_t<D>>
  class view_interface {
    […]

    constexpr explicit operator bool()
      requires empty_checkable_range<D>~~requires { ranges::empty(derived()); }~~ {
        return !ranges::empty(derived());
```

```
    }
  constexpr explicit operator bool() const
    requires empty_checkable_range<const D>requires { ranges::empty(derived()); } {
      return !ranges::empty(derived());
    }

  […]
};
[…]
}
```

5. Modify 26.7.6.2 [range.ref.view], class template `ref_view` synopsis, as indicated:

```
namespace std::ranges {
  template<range R>
    requires is_object_v<R>
  class ref_view : public view_interface<ref_view<R>> {
    […]

    constexpr bool empty() const
      requires empty_checkable_range<R>requires { ranges::empty(*r_); }
    { return ranges::empty(*r_); }

    […]
  };
  […]
}
```

6. Modify 26.7.6.3 [range.owning.view], class template `owning_view` synopsis, as indicated:

```
namespace std::ranges {
  template<range R>
    requires movable<R> && (!is-initializer-list<R>) // see [range.refinements]
  class owning_view : public view_interface<owning_view<R>> {
    […]

    constexpr bool empty() requires empty_checkable_range<R>requires { ranges::empty(r_); }
    { return ranges::empty(r_); }
    constexpr bool empty() const requires empty_checkable_range<const R>requires { ranges::empty(r_); }
    { return ranges::empty(r_); }

    […]
  };
  […]
}
```

7. Modify 26.7.7.2 [range.as.rvalue.view], class template `as_rvalue_view` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
    requires input_range<V>
  class as_rvalue_view : public view_interface<as_rvalue_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>
    { return ranges::empty(base_); }
    constexpr bool empty() const requires empty_checkable_range<const V>
    { return ranges::empty(base_); }

    constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
    constexpr auto size() const requires sized_range<const V> { return ranges::size(base_); }
  };
  […]
}
```

8. Modify 26.7.9.2 [range.transform.view], class template `transform_view` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V, move_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  class transform_view : public view_interface<transform_view<V, F>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>
    { return ranges::empty(base_); }
```

```
    constexpr bool empty() const requires empty_checkable_range<const V>
    { return ranges::empty(base_); }

    constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_); }
  };
  […]
}
```

9. Modify 26.7.10.2 [range.take.view], class template `take_view` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
  class take_view : public view_interface<take_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V> {
      return ranges::empty(base_) || count_ == 0;
    }
    constexpr bool empty() const requires empty_checkable_range<const V> {
      return ranges::empty(base_) || count_ == 0;
    }

    constexpr auto size() requires sized_range<V> {
      auto n = ranges::size(base_);
      return ranges::min(n, static_cast<decltype(n)>(count_));
    }
    constexpr auto size() const requires sized_range<const V> {
      auto n = ranges::size(base_);
      return ranges::min(n, static_cast<decltype(n)>(count_));
    }
  };
  […]
}
```

10. Modify 26.7.16.2 [range.lazy.split.view], class template `lazy_split_view` synopsis, as indicated:

```
namespace std::ranges {
  […]
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  class lazy_split_view : public view_interface<lazy_split_view<V, Pattern>> {
    […]
    constexpr bool empty() requires empty_checkable_range<V> &&
                                    empty_checkable_range<Pattern>
    { return ranges::empty(base_) && ranges::empty(pattern_); }

    constexpr bool empty() const requires empty_checkable_range<const V> &&
                                          empty_checkable_range<const Pattern>
    { return ranges::empty(base_) && ranges::empty(pattern_); }
  };
  […]
}
```

11. Modify 26.7.17.2 [range.split.view] as indicated:

```
constexpr subrange<iterator_t<V>> find-next(iterator_t<V> it);
```

-5- *Effects*: Equivalent to:

```
    auto [b, e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
    if (b != ranges::end(base_) && ranges::begin(pattern_) == ranges::end(pattern_)ranges::empty(pattern_)) {
      ++b;
      ++e;
    }
    return {b, e};
```

12. Modify 26.7.19.2 [range.common.view], class template `common_view` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
    requires (!common_range<V> && copyable<iterator_t<V>>)
  class common_view : public view_interface<common_view<V>> {
```

```
    […]

    constexpr bool empty() requires empty_checkable_range<V> {
      return ranges::empty(base_);
    }
    constexpr bool empty() const requires empty_checkable_range<const V> {
      return ranges::empty(base_);
    }

    constexpr auto size() requires sized_range<V> {
      return ranges::size(base_);
    }
    constexpr auto size() const requires sized_range<const V> {
      return ranges::size(base_);
    }
  };
  […]
}
```

13. Modify 26.7.21.2 [range.as.const.view], class template `as_const_view` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
    requires input_range<V>
  class as_const_view : public view_interface<as_const_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>
    { return ranges::empty(base_); }
    constexpr bool empty() const requires empty_checkable_range<const V>
    { return ranges::empty(base_); }

    constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
    constexpr auto size() const requires sized_range<const V> { return ranges::size(base_); }
  };
  […]
}
```

14. Modify 26.7.22.2 [range.elements.view], class template `elements_view` synopsis, as indicated:

```
namespace std::ranges {
  […]
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
             returnable-element<range_reference_t<V>, N>
  class elements_view : public view_interface<elements_view<V, N>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>
    { return ranges::empty(base_); }
    constexpr bool empty() const requires empty_checkable_range<const V>
    { return ranges::empty(base_); }

    constexpr auto size() requires sized_range<V>
    { return ranges::size(base_); }
    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_); }

    […]
  };
  […]
}
```

15. Modify 26.7.23.2 [range.enumerate.view], class template `enumerate_view` synopsis, as indicated:

```
namespace std::ranges {
  template<view V>
    requires range-with-movable-references<V>
  class enumerate_view : public view_interface<enumerate_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>
    { return ranges::empty(base_); }
    constexpr bool empty() const requires empty_checkable_range<const V>
    { return ranges::empty(base_); }

    constexpr auto size() requires sized_range<V>
```

```
    { return ranges::size(base_); }
    constexpr auto size() const requires sized_range<const V>
    { return ranges::size(base_); }

    […]
  };
  […]
}
```

16. Modify 26.7.24.2 [range.zip.view], class template `zip_view` synopsis, as indicated:

```
namespace std::ranges {
  […]
  template<input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0)
  class zip_view : public view_interface<zip_view<Views...>> {
    […]

    constexpr bool empty() requires (empty_checkable_range<Views> && ...);
    constexpr bool empty() requires (empty_checkable_range<const Views> && ...);

    constexpr auto size() requires (sized_range<Views> && ...);
    constexpr auto size() const requires (sized_range<const Views> && ...);

  };
  […]
}
```

[…]

```
  constexpr bool empty() requires (empty_checkable_range<Views> && ...);
  constexpr bool empty() requires (empty_checkable_range<const Views> && ...);
```

-?- *Effects*: Equivalent to:

```
    return apply([](auto... is_empty) { return (is_empty || ...); },
                 tuple-transform(ranges::empty, views_));
```

```
constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
```

-3- *Effects*: Equivalent to:

```
    return apply([](auto... sizes) {
      using CT = make-unsigned-like-t<common_type_t<decltype(sizes)...>>;
      return ranges::min({CT(sizes)...});
    }, tuple-transform(ranges::size, views_));
```

17. Modify 26.7.25.2 [range.zip.transform.view], class template `zip_transform_view` synopsis, as indicated:

```
namespace std::ranges {
  template<move_constructible F, input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<Views>...> &&
             can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
  class zip_transform_view : public view_interface<zip_transform_view<F, Views...>> {
    […]

    constexpr bool empty() requires empty_checkable_range<InnerView> {
      return zip_.empty();
    }

    constexpr bool empty() const requires empty_checkable_range<const InnerView> {
      return zip_.empty();
    }

    constexpr auto size() requires sized_range<InnerView> {
      return zip_.size();
    }

    constexpr auto size() const requires sized_range<const InnerView> {
      return zip_.size();
    }
  };
  […]
}
```

18. Modify 26.7.28.2 [range.chunk.view.input], class template `chunk_view` synopsis for input ranges, as indicated:

```
namespace std::ranges {
  […]
  template<view V>
    requires input_range<V>
  class chunk_view : public view_interface<chunk_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<Vs>;
    constexpr bool empty() const requires empty_checkable_range<const V>;

    constexpr auto size() requires sized_range<V>;
    constexpr auto size() const requires sized_range<const V>;
  };
  […]
}
```

[…]

```
  constexpr bool empty() requires empty_checkable_range<Vs>;
  constexpr bool empty() const requires empty_checkable_range<const V>;
```

-?- *Effects*: Equivalent to:

```
    return ranges::empty(base_);
```

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

-5- *Effects*: Equivalent to:

```
    return to-unsigned-like(div-ceil(ranges::distance(base_), n_));
```

19. Modify 26.7.31.2 [range.stride.view], class template `stride_view` synopsis, as indicated:

```
namespace std::ranges {
  template<input_range V>
    requires view<V>
  class stride_view : public view_interface<stride_view<V>> {
    […]

    constexpr bool empty() requires empty_checkable_range<V>;
    constexpr bool empty() const requires empty_checkable_range<const V>;

    constexpr auto size() requires sized_range<V>;
    constexpr auto size() const requires sized_range<const V>;
    […]
  };
  […]
}
```

[…]

```
  constexpr bool empty() requires empty_checkable_range<V>;
  constexpr bool empty() const requires empty_checkable_range<const V>;
```

-?- *Effects*: Equivalent to:

```
    return ranges::empty(base_);
```

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

-5- *Effects*: Equivalent to:

```
    return to-unsigned-like(div-ceil(ranges::distance(base_), stride_));
```

20. Modify 26.7.32.2 [range.cartesian.view], class template `cartesian_product_view` synopsis, as indicated:

```
namespace std::ranges {
  […]
  template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
  class cartesian_product_view : public view_interface<cartesian_product_view<First, Vs...>> {
    […]

    constexpr bool empty() requires (empty_checkable_range<First> && ... &&
```

```
                                        empty_checkable_range<Vs>);
    constexpr bool empty() const requires (empty_checkable_range<const First> && ... &&
                                           empty_checkable_range<const Vs>);

    constexpr see below size()
      requires cartesian-product-is-sized<First, Vs...>;
    constexpr see below size() const
      requires cartesian-product-is-sized<const First, const Vs...>;

    […]
  };
  […]
}
```

[…]

```
constexpr iterator<false> end()
  requires ((!simple-view<First> || ... || !simple-view<Vs>)
    && cartesian-product-is-common<First, Vs...>);
constexpr iterator<true> end() const
  requires cartesian-product-is-common<const First, const Vs...>;
```

-4- Let:

(4.1) — *is-const* be `true` for the const-qualified overload, and `false` otherwise;

(4.2) — *is-empty* be `true` if the expression `ranges::begin(rng) == ranges::end(rng)`~~`ranges::empty(rng)`~~ is `true` for any `rng` among the underlying ranges except the first one and `false` otherwise; and

(4.3) — *begin-or-first-end*`(rng)` be expression-equivalent to *is-empty* `? ranges::begin(rng) :` *cartesian-common-arg-end*`(rng)` if `rng` is the first underlying range and `ranges::begin(rng)` otherwise.

-5- *Effects*: Equivalent to:

```
iterator<is-const> it(tuple-transform(
  [](auto& rng){ return begin-or-first-end(rng); }, bases_));
return it;
```

```
constexpr default_sentinel_t end() const noexcept;
```

-6- *Returns*: `default_sentinel`.

```
  constexpr bool empty() requires (empty_checkable_range<First> && ... &&
                                   empty_checkable_range<Vs>);
  constexpr bool empty() const requires (empty_checkable_range<const First> && ... &&
                                         empty_checkable_range<const Vs>);
```

-?- *Effects*: Equivalent to:

```
    return apply([](auto... is_empty) { return (is_empty || ...); },
                 tuple-transform(ranges::empty, bases_));
```